

CASE STUDY #1

React Dashboard Performance Optimization

SupportDesk Ticketing Platform

PROJECT OVERVIEW

SupportDesk is a React/Next.js-based ticketing platform designed to help support teams manage customer issues efficiently.

Core capabilities include:

 Ticket Management

 Ticket Search

 Advanced Filtering

  Agent Assignment

 Support Operations Tracking

As the amount of ticket data increased, the Tickets page began experiencing noticeable performance issues that negatively impacted the user experience.

THE PROBLEM

Users reported that the Tickets page felt slow and unresponsive.

Common complaints included:

 "Search feels laggy."

 "Filters take time to apply."

 "The page takes too long to load."

 "The dashboard feels heavy."

The goal of this investigation was to identify the root causes of these issues and improve the responsiveness of the application.

Tickets Dashboard

SupportDesk

Ticketing & Support Dashboard

Search (todo) HT

Overview

Tickets

Board

Analytics

Agents

Settings

Tickets

Browse, filter, and manage all support tickets in one place.

All statuses

All priorities

All assignees

Search by ID, title, requester... 10000 of 10000

ID	TITLE	STATUS	PRIORITY	ASSIGNEE	CREATED	UPDATED	TAGS
#1002-COPY-1	Unable to login via Google copy 1 Priya Shah 1 · requester1@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-25	Unable to login via Google copy 25 Priya Shah 25 · requester25@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-49	Unable to login via Google copy 49 Priya Shah 49 · requester49@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-73	Unable to login via Google copy 73 Priya Shah 73 · requester73@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-97	Unable to login via Google copy 97 Priya Shah 97 · requester97@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-121	Unable to login via Google copy 121 Priya Shah 121 · requester121@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-145	Unable to login via Google copy 145 Priya Shah 145 · requester145@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-169	Unable to login via Google copy 169 Priya Shah 169 · requester169@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-193	Unable to login via Google copy 193 Priya Shah 193 · requester193@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth
#1002-COPY-217	Unable to login via Google copy 217 Priya Shah 217 · requester217@example.com	In progress	High	Sarah Coleman	Jan 11, 02:50 PM	Jan 12, 08:15 PM	auth, google-oauth

Search & Filter Experience

Observed Issues:

 Search lag

 Filter lag

 Delayed UI updates

 Heavy page rendering

Before:

 [SupportDesk – Before Optimization Demo](#)

<https://drive.google.com/file/d/1NqHumYcf0cJX6y-LKTih93OrfBlqAekC/view?usp=sharing>

After (Post Performance Optimization):

 [SupportDesk – After Optimization Demo](#)

<https://drive.google.com/file/d/1XurDFOQUs8Hdh220oKKPE6laVcbgg4Yp/view?usp=sharing>

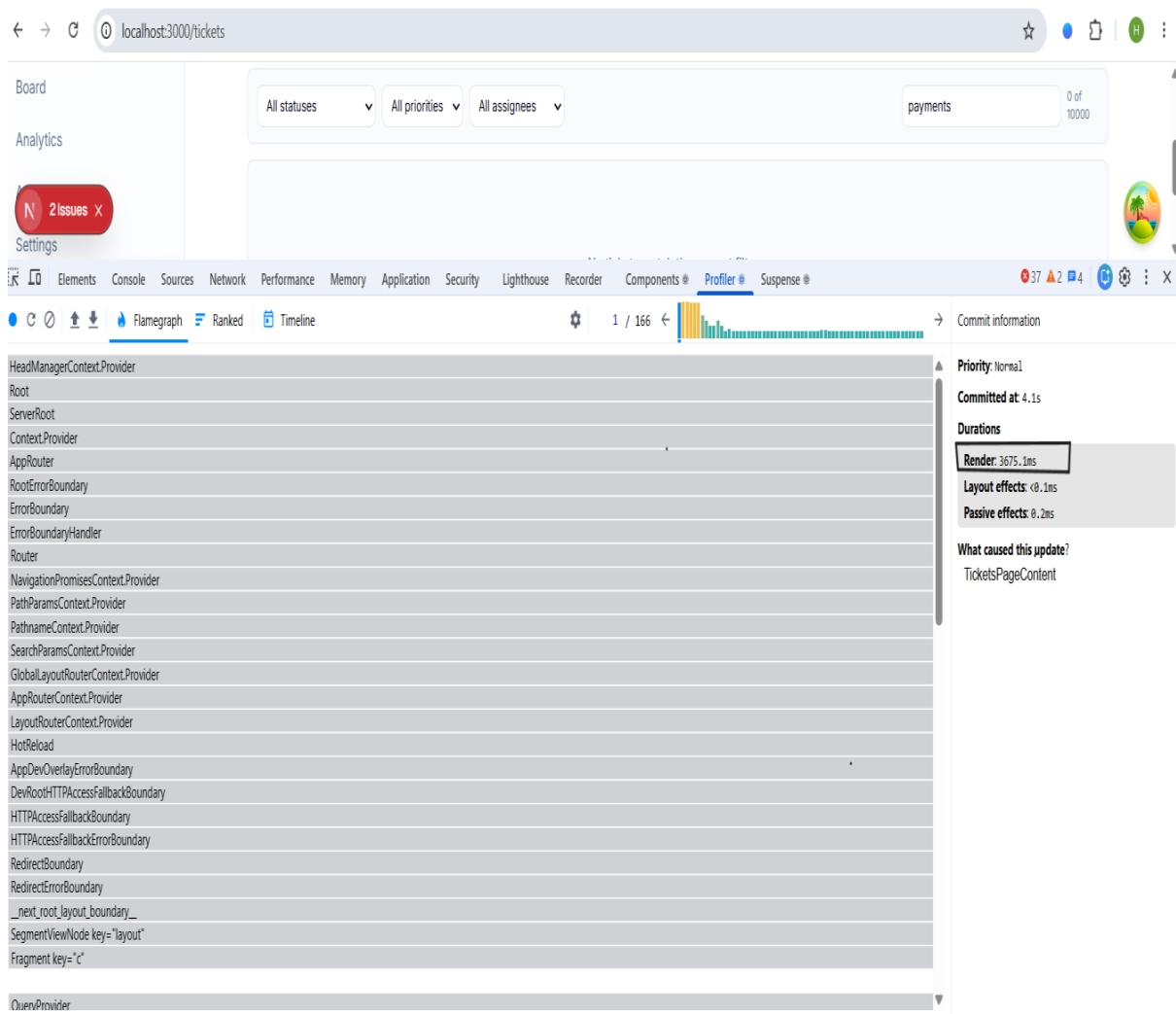
React Profiler Analysis

Initial React Profiler recording showed:

 Render Time: 3675ms

This immediately confirmed that rendering work was extremely expensive.

Before (Render Time = 3675ms)



After (Render Time = 2.3 ms)

Agents

Settings

N

ID	TITLE	STATUS	PRIORITY	ASSIGNEE	CREATED	UPDATED	TAGS
#1001-COPY-2	Payment failed for premium plan copy 2 Raj Malhotra 2 - requester2@example.com	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...
#1001-COPY-26	Payment failed for premium plan copy 26 Raj Malhotra 26 - requester26@example.com	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...
#1001-COPY-50	Payment failed for premium plan copy 50	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Components Profiler Suspense

Flamegraph Ranked Timeline

1 / 17

Commit information

Priority: Immediate

Committed at: 14.8s

Durations

Render: 2.3ms

Layout effects: <0.1ms

Passive effects: 0.2ms

What caused this update?

TicketFilters

HeadManagerContext.Provider

Root

ServerRoot

Context.Provider

AppRouter

RootErrorBoundary

ErrorBoundary

ErrorBoundaryHandler

Router

NavigationPromisesContext.Provider

PathParamsContext.Provider

PathnameContext.Provider

SearchParamsContext.Provider

GlobalLayoutRouterContext.Provider

AppRouterContext.Provider

LayoutRouterContext.Provider

HotReload

AppDevOverlayErrorBoundary

DevRootHTTPAccessFallbackBoundary

HTTPAccessFallbackBoundary

HTTPAccessFallbackErrorBoundary

RedirectBoundary

RedirectErrorBoundary

_next_root_layout_boundary_

SegmentViewNode key="layout"

Fragment key="c"

QueryProvider

INVESTIGATION APPROACH

Instead of immediately applying fixes, the investigation followed a structured performance engineering process.

Problem



Evidence Collection



Diagnosis



Optimization



Validation

This approach ensured that every optimization was backed by measurable evidence.

ROOT CAUSE ANALYSIS

The investigation identified three major performance bottlenecks.

PROBLEM #1

Slow Initial Page Render

Observation

The Tickets page itself took a long time to become usable.

Root Cause

The application generated a dataset containing:

 10,000 tickets

during component rendering.

This work was repeated unnecessarily and significantly increased rendering cost.

Impact

 High CPU usage

 Large memory allocations

 Expensive React rendering

 Slow initial page load

PROBLEM #2

Slow Filter Performance

Observation

Changing ticket filters caused noticeable delays.

Root Cause

Each filter action triggered:

Filter 10,000 Tickets



Sort Results



Re-render Ticket Table

This caused excessive work for every filter interaction.

Impact

 Delayed filter response

 Unnecessary computations

 Excessive rerenders

PROBLEM #3

Slow Search Performance

Observation

Typing into the search box felt sluggish.

Root Cause

Search logic executed on every keystroke.

Example:

p

pa

pay

paym

payme

payment

Each keystroke triggered:

 Full dataset scan

 Sorting operations

 React updates

Additionally:

- ✗ No memoization
- ✗ Expensive string processing
- ✗ Artificial CPU-heavy loops

Impact

 Typing lag

 Search delays

 Increased rendering cost

OPTIMIZATION IMPLEMENTED

Several optimizations were introduced to address the identified bottlenecks.

OPTIMIZATION #1

Memoized Dataset Generation

Before

Generate 10,000 Tickets

Every Render

After

Generate Once

Reuse Later

Benefit

 Eliminated repeated dataset creation

 Reduced CPU usage

 Reduced rendering workload

OPTIMIZATION #2

Memoized Filtering Logic

Before

Recompute Everything

Every Render

After

Compute Only When Needed

Benefit

 Faster filters

 Reduced unnecessary computations

 Lower rendering cost

OPTIMIZATION #3

Search Optimization

Implemented:

 Debounced Search

 useDeferredValue

✓ Precomputed Search Text

Benefit

- ⚡ Smooth typing experience
 - ⚡ Reduced search workload
 - ⚡ Better responsiveness
-

⚡ OPTIMIZATION #4

Removed Unnecessary CPU Work

Removed intentionally expensive search loops and redundant computations.

Benefit

- ⚡ Lower CPU consumption
 - ⚡ Faster updates
 - ⚡ Better overall performance
-

⚡ OPTIMIZATION #5

Table Pagination

Before

Render 10,000 Rows

After

Render 50 Rows Per Page

Benefit

⚡ Smaller DOM

⚡ Faster rendering

⚡ Better scalability

⚡ Improved user experience

RESULTS

React Profiler Comparison

Before

 Render Time: 3675ms

After

 Render Time: 2.3ms

PERFORMANCE IMPROVEMENT

3675ms → 2.3ms

Approximately:

 **1,600x Faster Rendering**



BEFORE VS AFTER React PROFILER

Before (Render Time: 3675ms)

localhost:3000/tickets

Board

Analytics

2 Issues

Settings

All statuses All priorities All assignees payments 0 of 10000

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Components Profiler Suspense

Flamegraph Ranked Timeline

1 / 166

Commit information

Priority: Normal

Committed at: 4.1s

Durations

Render: 3675.1ms

Layout effects: <0.1ms

Passive effects: 0.2ms

What caused this update?

TicketsPageContent

HeadManagerContext.Provider

Root

ServerRoot

Context.Provider

AppRouter

RootErrorBoundary

ErrorBoundary

ErrorBoundaryHandler

Router

NavigationPromisesContext.Provider

PathParamsContext.Provider

PathnameContext.Provider

SearchParamsContext.Provider

GlobalLayoutRouterContext.Provider

AppRouterContext.Provider

LayoutRouterContext.Provider

HotReload

AppDevOverlayErrorBoundary

DevRootHTTPAccessFallbackBoundary

HTTPAccessFallbackBoundary

HTTPAccessFallbackErrorBoundary

RedirectBoundary

RedirectErrorBoundary

__next_root_layout_boundary__

SegmentViewNode key="layout"

Fragment key="c"

ThemeProvider



After (Render Time: 2.3ms)

localhost:3000/tickets

Agents
Settings

N

ID	TITLE	STATUS	PRIORITY	ASSIGNEE	CREATED	UPDATED	TAGS
#1001-COPY-2	Payment failed for premium plan copy 2 Raj Malhotra 2 - requester2@example.com	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...
#1001-COPY-26	Payment failed for premium plan copy 26 Raj Malhotra 26 - requester26@example.com	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...
#1001-COPY-50	Payment failed for premium plan copy 50	Open	Urgent	John Doe	Jan 12, 04:00 PM	Jan 12, 04:30 PM	billing, payments...

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder Components Profiler Suspense

Flamegraph Ranked Timeline

1 / 17

Commit information

HeadManagerContext.Provider
Root
ServerRoot
Context.Provider
AppRouter
RootErrorBoundary
ErrorBoundary
ErrorBoundaryHandler
Router
NavigationPromisesContext.Provider
PathParamsContext.Provider
PathnameContext.Provider
SearchParamsContext.Provider
GlobalLayoutRouterContext.Provider
AppRouterContext.Provider
LayoutRouterContext.Provider
HotReload
AppDevOverlayErrorBoundary
DevRootHTTPAccessFallbackBoundary
HTTPAccessFallbackBoundary
HTTPAccessFallbackErrorBoundary
RedirectBoundary
RedirectErrorBoundary
_next_root_layout_boundary_
SegmentViewNode key= "layout"
Fragment key= "c"
QueryProvider

Priority: Immediate

Committed at: 14.8s

Durations

Render: 2.3ms

Layout effects: <0.1ms

Passive effects: 0.2ms

What caused this update?

TicketFilters

 RESULTS SUMMARY

Metric	Before	After
Render Time	3675ms	2.3ms
Search Experience	Laggy	Smooth
Filter Response	Slow	Fast
Rows Rendered	10,000	50
CPU Usage	High	Lower
Responsiveness	Poor	Excellent

USER EXPERIENCE IMPACT

Before

 Slow page load

 Search lag

 Filter lag

 Heavy UI

 Poor responsiveness

After

 Faster page load

 Responsive search

 Instant filters

 Smooth interactions

 Better user experience

BUSINESS IMPACT

Frontend performance directly affects:

-  User productivity
-  Application responsiveness
-  Customer satisfaction
-  Scalability

By reducing rendering work and optimizing search and filtering operations, the application became significantly more responsive and capable of handling larger datasets efficiently.

TECHNIQUES USED

-  React Profiler
-  useMemo
-  useDeferredValue
-  Debounced Search
-  Memoized Computation
-  Pagination
-  Render Optimization

Large Dataset Optimization

KEY LESSONS

Performance optimization is rarely about one magic fix.

The biggest gains usually come from:

- ☒ Measuring first
 - ☒ Identifying bottlenecks
 - ☒ Removing unnecessary work
 - ☒ Rendering less
 - ☒ Computing less
 - ☒ Validating results with evidence
-

FINAL OUTCOME

A slow React ticketing dashboard was transformed into a highly responsive application through systematic performance analysis, React profiling, memoization, search optimization, and rendering optimization.

The result was a measurable improvement from:

 3675ms Render Time

to

 2.3ms Render Time

while maintaining the same functionality and improving the overall user experience.



Before:

 [SupportDesk – Before Optimization Demo](https://drive.google.com/file/d/1NqHumYcf0cJX6y-LKTih93OrfBlqAekC/view?usp=sharing)
<https://drive.google.com/file/d/1NqHumYcf0cJX6y-LKTih93OrfBlqAekC/view?usp=sharing>



After (Post Performance Optimization):

 [SupportDesk – After Optimization Demo](https://drive.google.com/file/d/1XurDFOQUs8Hdh220oKKPE6laVcbgg4Yp/view?usp=sharing)
<https://drive.google.com/file/d/1XurDFOQUs8Hdh220oKKPE6laVcbgg4Yp/view?usp=sharing>
